

Explain Good Leaf Node Pairs

Let's solve this **step by step**. The main trick is:

At every node, we want to know **how far the leaf nodes below it are**.

We use **DFS + postorder traversal**.

1. Understand the problem

Suppose we have this tree:



Leaves are:

3 and 4

The shortest path is:

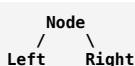
4 → 2 → 1 → 3

Distance = 3.

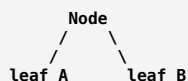
So (3, 4) is a **good pair**.

2. The important observation

Consider a node:



Suppose there are leaf nodes below both sides:



If:

- distance from **Node** to leaf A = **a**
- distance from **Node** to leaf B = **b**

Then the distance between the two leaves is:

a + b

So the pair is good if:

a + b <= distance

That is the entire core idea.

3. What should DFS return?

For every node, DFS returns:

A list/array containing the distances from the current node to all leaf nodes in its subtree.

For example:



Leaves are 4, 5, 3.

At node 2:



Distances from node 2:

4 → distance 1
5 → distance 1

So DFS for node 2 returns:

```
C++
[1, 1]
```

At node 1, leaf 3 is at distance 1.

The leaves 4 and 5 are one level further:

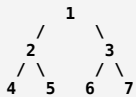
```
distance(1, 4) = 2
distance(1, 5) = 2
```

So node 1 returns:

```
C++
[2, 2, 1]
```

4. Visualize the algorithm

Take Example 2:



Leaves:

4, 5, 6, 7

Distance = 3.

Let's process from the bottom upward.

Step A: Leaf nodes

For a leaf:

4

Distance from itself to itself is:

0

So it returns:

```
C++
[0]
```

Similarly:

```
4 → [0]
5 → [0]
6 → [0]
7 → [0]
```

5. Process node 2



Left DFS:

```
C++
[0]
```

Right DFS:

```
C++
[0]
```

Now check every left leaf with every right leaf:

```
distance = 0 + 0 + 2
```

Why +2?

Because the path is:

```
4 → 2 → 5
```

Two edges.

```
0 + 0 + 2 = 2
```

Since:

```
2 <= 3
```

This is a good pair.

```
(4, 5) ✓
```

Answer becomes:

```
count = 1
```

Now we return distances from node 2.

Leaf 4 was distance 0 from itself, but from node 2:

```
0 + 1 = 1
```

Same for 5.

So node 2 returns:

```
C++
[1, 1]
```

6. Process node 3

```

  3
 / \
6   7
```

Same process:

```
C++
left = [0]
right = [0]
```

Distance:

```
0 + 0 + 2 = 2
```

Good pair:

```
(6, 7) ✓
```

Now:

```
count = 2
```

Node 3 returns:

```
C++
[1, 1]
```

7. Process root node 1

Now we have:

```

      1
     / \
    2   3

DFS(2) = [1, 1]
DFS(3) = [1, 1]
```

Each 1 means:

There is a leaf at distance 1 from node 2 or 3.

But from root 1, the distance between a leaf from the left and one from the right is:

`leftDistance + rightDistance + 2`

For example (4, 6):

`4 → 2 → 1 → 3 → 6`

Distance:

`1 + 1 + 2 = 4`

Since:

`4 > 3`

Not a good pair.

Same for all cross pairs:

(4,6) ✗
 (4,7) ✗
 (5,6) ✗
 (5,7) ✗

Final answer:

2

The easiest approach

We can write DFS like this:

```

DFS(node):
    If node is NULL:
        return empty list

    If node is a leaf:
        return [0]

    leftDistances = DFS(node.left)
    rightDistances = DFS(node.right)

    For every left distance:
        For every right distance:
            if left + right + 2 <= distance:
                count++

    Create result

    Add 1 to every distance from left
    Add 1 to every distance from right

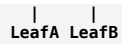
    Return result
```

Important: Why +2?

Suppose:

```

Current Node
  /   \
...   ...
```



The DFS values represent distances from the **children**.

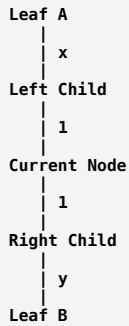
If:

```

left distance = x
right distance = y

```

To go from Leaf A to Leaf B:



Therefore:

```

total = x + 1 + 1 + y

```

So:

```

C++
x + y + 2

```

Simple C++ code

```

C++
class Solution {
public:
    int ans = 0;
    int maxDistance;

    vector<int> dfs(TreeNode* node) {
        // Base case: no node
        if (node == nullptr) {
            return {};
        }

        // Base case: leaf node
        if (node->left == nullptr && node->right == nullptr) {
            return {0};
        }

        // Get leaf distances from left and right subtree
        vector<int> left = dfs(node->left);
        vector<int> right = dfs(node->right);

        // Check pairs where one leaf is in left
        // and another leaf is in right
        for (int l : left) {
            for (int r : right) {
                if (l + r + 2 <= maxDistance) {
                    ans++;
                }
            }
        }

        vector<int> result;

        // Move one level up
        for (int l : left) {
            result.push_back(l + 1);
        }

        for (int r : right) {
            result.push_back(r + 1);
        }

        return result;
    }

    int countPairs(TreeNode* root, int distance) {
        maxDistance = distance;
        dfs(root);
        return ans;
    }
}

```

```
}; }
```

Let's trace the code on Example 1



Distance:

```
3
```

DFS(4)

4 is a leaf:

```
C++
return {0};
```

DFS(2)

Left:

```
C++
{}
```

Right:

```
C++
{0}
```

There is no pair because one side is empty.

Move upward:

```
C++
{0 + 1}
```

So:

```
C++
DFS(2) = {1}
```

DFS(3)

Leaf:

```
C++
DFS(3) = {0}
```

DFS(1)

Now:

```
C++
left = {1}
right = {0}
```

Check:

```
C++
1 + 0 + 2 = 3
```

Since:

```
3 <= 3
```

Good pair!

```
ans = 1
```

Final answer:

1

Why do we use Postorder DFS?

The traversal is basically:

```
Left subtree
  ↓
Right subtree
  ↓
Current node
```

Because before processing the current node, we need to know:

```
What leaf nodes exist in my left subtree?
What leaf nodes exist in my right subtree?
How far are they from me?
```

So we first process:

children

Then:

current node

This is exactly **postorder DFS**.

Time Complexity

There are at most:

210 nodes

And:

distance <= 10

So this approach is easily fast enough.

A simple rough complexity is:

$O(N \times \text{distance}^2)$

because we only really care about leaf distances up to the allowed distance.

For the given constraints, this is very efficient.

One improvement: only keep useful distances

Suppose:

distance = 3

If a leaf is already farther than 3 from the current node, it can never form a valid pair higher up.

So we can ignore it.

Here is a slightly optimized version:

```
C++
class Solution {
public:
    int ans = 0;
    int maxDistance;

    vector<int> dfs(TreeNode* node) {
        if (node == nullptr) {
            return {};
        }

        // If it is a leaf
        if (node->left == nullptr && node->right == nullptr) {
            return {0};
        }
    }
}
```

```

vector<int> left = dfs(node->left);
vector<int> right = dfs(node->right);

// Count valid pairs
for (int l : left) {
    for (int r : right) {
        if (l + r + 2 <= maxDistance) {
            ans++;
        }
    }
}

vector<int> result;

// Return distances for parent
for (int l : left) {
    if (l + 1 < maxDistance) {
        result.push_back(l + 1);
    }
}

for (int r : right) {
    if (r + 1 < maxDistance) {
        result.push_back(r + 1);
    }
}

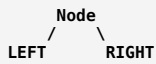
return result;
}

int countPairs(TreeNode* root, int distance) {
    maxDistance = distance;
    dfs(root);
    return ans;
}
};

```

The key idea to remember

For each node:



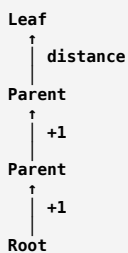
1. Recursively get distances to leaf nodes.
2. Compare every leaf from the left with every leaf from the right.
3. If:

```
leftDistance + rightDistance + 2 <= distance
```

count it.

4. Add 1 to every distance and return them to the parent.

Mental model



So the DFS continuously carries "**distance from this subtree's leaves to the current node**" upward.

This is the cleanest approach for understanding and writing the solution in an interview or LeetCode.

Yes — **this part is very important**, because this is how we send the information **from the current node to its parent**.

Let's focus only on this code:

```

C++
vector<int> result;

// Return distances for parent
for (int l : left) {
    if (l + 1 < maxDistance) {
        result.push_back(l + 1);
    }
}

```

```
for (int r : right) {
    if (r + 1 < maxDistance) {
        result.push_back(r + 1);
    }
}

return result;
```

First understand: What is inside left and right?

Suppose we are currently at this node:



Imagine we are currently processing node 2.



Both 4 and 5 are leaf nodes.

When DFS reaches a leaf, we return:

```
C++
return {0};
```

Why 0?

Because the distance from the leaf to **itself** is 0.

So:

```
DFS(4) → {0}
DFS(5) → {0}
```

Now we are back at node 2.

Therefore:

```
C++
left = {0};
right = {0};
```

This means:

```
left = distances from node 4's subtree
right = distances from node 5's subtree
```

Step 1: We have already counted pairs

At node 2:



We check:

```
C++
l + r + 2 <= maxDistance
```

Here:

```
l = 0
r = 0
```

So:

```
0 + 0 + 2 = 2
```

That counts the pair (4, 5).

Now the pair counting is finished.

The next question is:

What information should node 2 send to its parent, node 1?

Step 2: Node 2 must return leaf distances to node 1

Look at the tree:



From node 2:

```
distance(2, 4) = 1
distance(2, 5) = 1
```

So node 2 should return:

```
C++
{1, 1}
```

This is exactly what this code does.

Dry run of the first loop

We have:

```
C++
left = {0};
```

Code:

```
C++
for (int l : left) {
    if (l + 1 < maxDistance) {
        result.push_back(l + 1);
    }
}
```

Let's say:

```
maxDistance = 3
```

Initially:

```
C++
result = {};
```

Now loop:

Iteration 1

```
C++
l = 0
```

Calculate:

```
C++
l + 1
```

So:

```
0 + 1 = 1
```

Check:

```
1 < 3
```

Yes.

Therefore:

```
C++
result.push_back(1);
```

Now:

```
C++
result = {1};
```

Why did we add 1?

Because we're moving **one level upward**.

Before:

```
DFS(4) returned distance from 4 to leaf 4
4 → 4 = 0
```

But now node 2 wants the distance:

```
2 → 4 = 1
```

So:

```
0 + 1 = 1
```

Now dry run of the second loop

We have:

```
C++
right = {0};
```

Code:

```
C++
for (int r : right) {
    if (r + 1 < maxDistance) {
        result.push_back(r + 1);
    }
}
```

Initially:

```
C++
result = {1};
```

Now:

```
C++
r = 0
```

Calculate:

```
r + 1 = 1
```

Check:

```
1 < 3
```

Yes.

So:

```
C++
result.push_back(1);
```

Now:

```
C++
result = {1, 1};
```

Finally:

```
C++
return result;
```

So:

```
DFS(2) returns {1, 1}
```

Visual meaning of {1, 1}



When **DFS(2)** returns:

```
C++
{1, 1}
```

It is telling node 1:

"I have two leaf nodes below me, and both are at distance 1 from me."

Those leaves are:

```
4 → distance 1 from node 2
5 → distance 1 from node 2
```

Now let's go one level higher

Suppose we are at node 1:



We already got:

```
C++
DFS(2) = {1, 1}
```

Node 3 is a leaf:

```
C++
DFS(3) = {0}
```

Therefore at node 1:

```
C++
left = {1, 1}
right = {0}
```

Let's understand the meaning:

```
left = {1, 1}
Node 2 → Leaf 4 = 1
Node 2 → Leaf 5 = 1
```

And:

```
right = {0}
Node 3 is itself a leaf.
```

Now why add 1 again?

After processing node 1, if node 1 had a parent, we would need to return distances from 1 to its leaf nodes.

For leaf 4:



4 /

From node 2:

$2 \rightarrow 4 = 1$

But from node 1:

$1 \rightarrow 2 \rightarrow 4 = 2$

Therefore:

$1 + 1 = 2$

So:

```
C++
left = {1, 1}
```

becomes:

```
C++
{2, 2}
```

The code:

```
C++
for (int l : left) {
    result.push_back(l + 1);
}
```

Dry run

Initially:

```
C++
result = {};
```

First value:

```
l = 1
l + 1 = 2
```

Result:

```
C++
result = {2};
```

Second value:

```
l = 1
l + 1 = 2
```

Result:

```
C++
result = {2, 2};
```

What about the right side?

```
C++
right = {};
```

This represents leaf 3.

From node 1 to node 3:

$1 \rightarrow 3 = 1$

So:

```
0 + 1 = 1
```

Therefore:

```
C++
result = {2, 2, 1};
```

Meaning:

```
From node 1:
Leaf 4 → distance 2
Leaf 5 → distance 2
Leaf 3 → distance 1
```

Complete dry run in table form

For this tree:



DFS returns:

Node	Is leaf?	Returned value	Meaning
4	Yes	{0}	Distance from 4 to itself = 0
5	Yes	{0}	Distance from 5 to itself = 0
2	No	{1, 1}	Leaves 4 and 5 are distance 1
3	Yes	{0}	Distance from 3 to itself = 0
1	No	{2, 2, 1}	Leaves 4,5,3 have distances 2,2,1

So the values are moving like this:

```
Leaf level:
4 → {0}
5 → {0}
3 → {0}

Move to node 2:
{0} + 1 → {1}
{0} + 1 → {1}
DFS(2) = {1, 1}

Move to node 1:
{1, 1} + 1 → {2, 2}
{0} + 1 → {1}
DFS(1) = {2, 2, 1}
```

Now understand the if condition

The optimized code has:

```
C++
if (l + 1 < maxDistance)
```

For example:

```
maxDistance = 3
```

Suppose:

```
C++
left = {0, 1, 2, 3}
```

Let's dry run.

```
l = 0
```

```
0 + 1 = 1
1 < 3 → true
```

Keep it:

```
C++
result.push_back(1);
```

$l = 1$

```
1 + 1 = 2
2 < 3 → true
```

Keep it:

```
C++
result.push_back(2);
```

$l = 2$

```
2 + 1 = 3
3 < 3 → false
```

Don't keep it.

$l = 3$

```
3 + 1 = 4
4 < 3 → false
```

Don't keep it.

So:

```
C++
left  = {0, 1, 2, 3}
result = {1, 2}
```

Why don't we keep distance 3 or greater?

Suppose:

```
maxDistance = 3
```

Imagine a leaf is already at distance 3 from the current node:

```
Current Node
|
|---- 3 edges ---- Leaf
```

Now this information is sent to the parent.

The parent would be one edge further away:

```
Parent
|
| 1
|
Current Node
|
|---- 3 edges ---- Leaf
```

Distance from parent becomes:

```
3 + 1 = 4
```

Now this leaf can never form a pair with another leaf with distance ≤ 3 .

Because even if the other leaf were directly at the parent:

```
other distance = 0
```

Total would already be:

```
4 + 0 = 4
```

which is greater than:

```
distance = 3
```

So we discard it.

The most important mental model

Think of every returned vector as a message.

For example:

```
C++
DFS(node) → {1, 2, 2}
```

means:

"From this node, I can reach three leaf nodes. Their distances are 1, 2, and 2."

Then the parent receives this message.

But from the parent's perspective, every leaf is **one edge further away**.

So:

```
Child returns:
{1, 2, 2}
Parent converts it into:
{2, 3, 3}
```

That is exactly why we do:

```
C++
result.push_back(l + 1);
```

In one line

```
C++
result.push_back(l + 1);
```

means:

"This leaf was 1 distance away from my child, so it is 1 + 1 distance away from me."

And:

```
C++
return result;
```

means:

"Parent, here are the distances from me to all useful leaf nodes below me."

That's the core of how this DFS solution passes information **bottom** → **up** through the tree.